

JIT Compilation Pipeline

This diagram illustrates the JIT compilation pipeline from bytecode to native code.

Complete JIT Pipeline

```

flowchart TD
    A[Bytecode] --> B{Hot Path?}
    B -->|No| C[Continue Interpretation]
    B -->|Yes| D[Select Compilation Tier]
    D --> E{Tier Selection}
    E -->|Tier 1| F[Baseline Compiler]
    E -->|Tier 2| G[Optimizing Compiler]
    F --> H[Template-Based Codegen]
    H --> I[Baseline Native Code]
    G --> J[Bytecode to LLVM IR]
    J --> K[LLVM Optimization Pipeline]
    K --> L[LLVM Code Generation]
    L --> M[Optimized Native Code]
    I --> N[Code Cache]
    M --> N
    N --> O[Native Execution]
    C --> P[Profiler]
    O --> P
    P --> Q{Recompile?}
    Q -->|Yes| D
    Q -->|No| R[Continue Execution]

    style A fill:#e1f5ff
    style I fill:#fff4e1
    style M fill:#ffe1e1
    style O fill:#e1f5ff
  
```

Tier 1: Baseline Compilation

```

flowchart LR
    A[Bytecode] --> B[Instruction Analysis]
    B --> C[Template Selection]
    C --> D[Register Allocation]
    D --> E[Code Emission]
    E --> F[Linking]
    F --> G[Baseline Code]

    style A fill:#ffcccc
    style G fill:#ccffcc
  
```

Baseline Compilation Steps

1. Instruction Analysis

- Parse bytecode instructions
- Identify instruction patterns
- Determine stack effects

2. Template Selection

- Select code template for each instruction
- Simple 1:1 or 1:N mapping
- No complex optimizations

3. Register Allocation

- Simple register allocation
- Stack-based with register caching
- Minimal register pressure

4. Code Emission

- Generate native instructions
- Direct template instantiation
- Inline constants

5. Linking

- Link with runtime support
- Resolve external references
- Generate call stubs

Example: Baseline Compilation

Bytecode:	Template:	Native Code:
PUSH 5	mov reg, imm push reg	mov rax, 5 push rax
PUSH 3	mov reg, imm push reg	mov rax, 3 push rax
ADD	pop reg2 pop reg1 add reg1, reg2 push reg1	pop rbx pop rax add rax, rbx push rax
RETURN	pop reg ret	pop rax ret

Tier 2: Optimizing Compilation

```

flowchart TD
    A[Bytecode] --> B[IR Generation]
    B --> C[LLVM IR]

    C --> D[Optimization Pipeline]

    D --> E[Scalar Optimizations]
    E --> F[Loop Optimizations]
    F --> G[Interprocedural Opts]
    G --> H[Code Generation Opts]

    H --> I[Optimized LLVM IR]

    I --> J[Instruction Selection]
    J --> K[Register Allocation]
    K --> L[Instruction Scheduling]
    L --> M[Peephole Optimization]

    M --> N[Optimized Native Code]

    style A fill:#ffcccc
    style C fill:#ffffcc
    style I fill:#ccffcc
    style N fill:#ccccff

```

LLVM Optimization Passes

Scalar Optimizations

```

flowchart LR
    A[Input IR] --> B[Constant Propagation]
    B --> C[Dead Code Elimination]
    C --> D[CSE]
    D --> E[Strength Reduction]
    E --> F[Optimized IR]

```

Loop Optimizations

```

flowchart LR
    A[Input IR] --> B[Loop Invariant Code Motion]
    B --> C[Loop Unrolling]
    C --> D[Loop Vectorization]
    D --> E[Loop Fusion]
    E --> F[Optimized IR]

```

Interprocedural Optimizations

```

flowchart LR
    A[Input IR] --> B[Inlining]
    B --> C[Devirtualization]
    C --> D[Constant Propagation]
    D --> E[Dead Function Elimination]
    E --> F[Optimized IR]

```

Example: Optimizing Compilation

Bytecode:

PUSH 5

PUSH 3

ADD

RETURN

↓ IR Generation

LLVM IR (Initial):

```
define double @function() {
```

```
entry:
```

```
    %0 = alloca double
```

```
    %1 = alloca double
```

```
    store double 5.0, double* %0
```

```
    store double 3.0, double* %1
```

```
    %2 = load double, double* %0
```

```
    %3 = load double, double* %1
```

```
    %4 = fadd double %2, %3
```

```
    ret double %4
```

```
}
```

↓ Optimization

LLVM IR (Optimized):

```
define double @function() {
```

```
entry:
```

```
    ret double 8.0
```

```
}
```

↓ Code Generation

Native Code:

```
mov rax, 0x4020000000000000 ; 8.0 in IEEE 754
```

```
ret
```

Hot Path Detection

```

flowchart TD
    A[Function Execution] --> B[Increment Counter]
    B --> C{Counter >= Baseline Threshold?}

    C -->|No| D[Continue Interpretation]
    C -->|Yes| E{Already Compiled?}

    E -->|No| F[Compile to Baseline]
    E -->|Yes| G{Counter >= Optimized Threshold?}

    G -->|No| H[Execute Baseline Code]
    G -->|Yes| I{Already Optimized?}

    I -->|No| J[Compile to Optimized]
    I -->|Yes| K[Execute Optimized Code]

    F --> H
    J --> K

    D --> L[Update Profile]
    H --> L
    K --> L
  
```

Threshold Values

```

Tier 0 (Interpreter):
- Threshold: 0
- Action: Profile execution

Tier 1 (Baseline JIT):
- Threshold: 100 executions
- Action: Fast compilation

Tier 2 (Optimized JIT):
- Threshold: 1000 executions
- Action: Full optimization
  
```

Code Cache Management

```

flowchart TD
    A[Compiled Code] --> B{Cache Full?}
    B -->|No| C[Add to Cache]
    B -->|Yes| D[Eviction Policy]
    D --> E{Eviction Strategy}
    E -->|LRU| F[Evict Least Recently Used]
    E -->|LFU| G[Evict Least Frequently Used]
    E -->|Size| H[Evict Largest Code]
    F --> I[Free Space]
    G --> I
    H --> I
    I --> C
    C --> J[Cache Entry]
    J --> K[Native Execution]
  
```

Cache Structure

Code Cache:

Function ID: 1 Tier: Baseline Code Size: 256 bytes Execution Count: 150 Last Access: timestamp Native Code: [...]
Function ID: 2 Tier: Optimized Code Size: 512 bytes Execution Count: 5000 Last Access: timestamp Native Code: [...]
...

Performance Characteristics

Compilation Time

```
gantt
    title JIT Compilation Time
    dateFormat X
    axisFormat %L ms

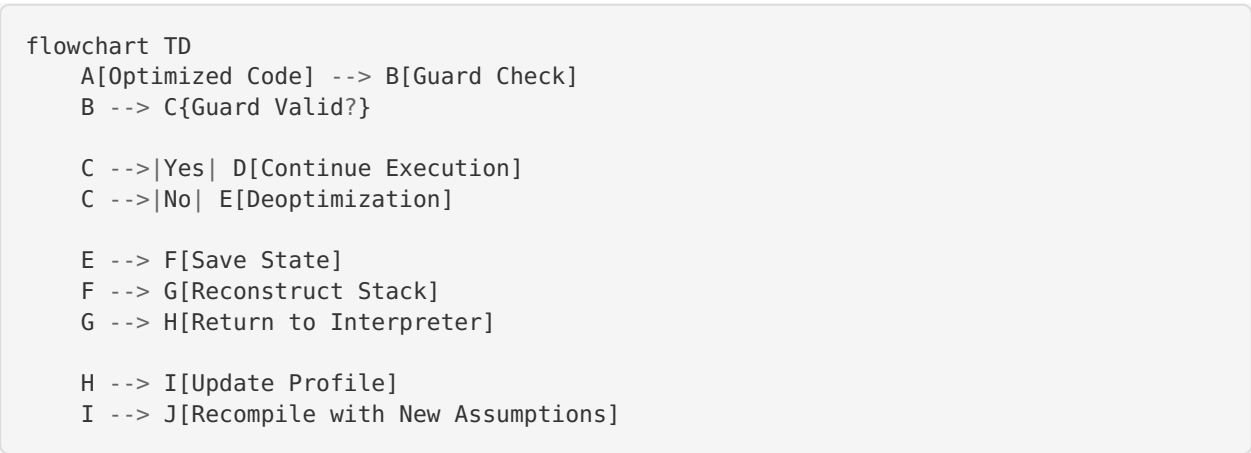
    section Baseline
    Analysis      :0, 1
    Template      :1, 2
    Register Alloc :2, 1
    Code Emission :3, 3
    Linking       :6, 2

    section Optimized
    IR Generation :0, 5
    Optimization  :5, 40
    Code Gen      :45, 30
    Linking       :75, 5
```

Execution Performance

Tier	Compilation Time	Execution Speed	Use Case
Interpreter	0 ms	1x (baseline)	Cold code
Baseline JIT	1-10 ms	5-10x	Warm code
Optimized JIT	10-100 ms	20-50x	Hot code

Deoptimization



Guard Types

- 1. **Type Guards:** Check object types
- 2. **Value Guards:** Check specific values
- 3. **Range Guards:** Check value ranges
- 4. **Null Guards:** Check for null pointers

[Back to Architecture Overview](#) (../README.md)